

LAPORAN PRAKTIKUM
Digital Input/Output pada ESP32 Menggunakan PlatformIO dan
Wokwi Simulator



Oleh:

NAMA : AKBAR BETHARA IMANI

NPM : 25782088

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026/2027

BAB I. PENDAHULUAN

1.1 Latar Belakang

Mikrokontroler ESP32 banyak digunakan dalam pengembangan sistem Internet of Things (IoT) karena memiliki dukungan konektivitas Wi-Fi dan Bluetooth bawaan, serta jumlah pin GPIO yang cukup banyak untuk berinteraksi dengan berbagai komponen elektronik. Praktikum ini bertujuan untuk memahami dasar interaksi ESP32 dengan komponen output (LED) dan input (push button) melalui pemrograman digital sederhana.

1.2 Tujuan Praktikum

- Memahami cara memprogram ESP32 untuk mengendalikan output digital (menyalakan/mematikan LED).
- Memahami cara membaca status input digital dari sebuah push button.
- Mampu merangkai dan mensimulasikan rangkaian mikrokontroler menggunakan Wokwi Simulator di dalam Visual Studio Code, terintegrasi dengan PlatformIO.

1.3 Alat dan Bahan

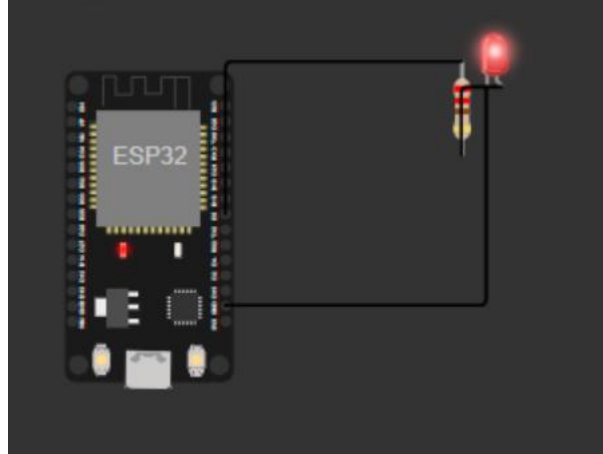
- 1 unit ESP32 DevKit V1 (disimulasikan)
- 1 buah LED merah
- 1 buah resistor 220 Ω (pembatas arus LED)
- 1 buah push button
- 1 buah resistor 10 k Ω (pull-down)
- Breadboard dan kabel jumper
- Visual Studio Code + ekstensi PlatformIO + ekstensi Wokwi Simulator

BAB II. PRAKTIKUM 1 — DIGITAL OUTPUT (MENYALAKAN LED)

2.1 Tujuan

Memprogram ESP32 agar dapat mengalirkan tegangan HIGH (3.3V) secara terprogram untuk menyalakan LED secara berkala.

2.2 Rangkaian



Gambar 2.1 Rangkaian LED pada Praktikum 1

2.3 Langkah Kerja

1. Pasang mikrokontroler pada breadboard.
2. Pasang LED pada baris kosong di breadboard. Kaki lebih panjang (Anoda) dihubungkan ke sisi positif, kaki lebih pendek (Katoda) ke sisi negatif.
3. Hubungkan pin D1 (GPIO 5) ke kaki Anoda LED menggunakan kabel jumper.
4. Hubungkan kaki Katoda LED ke salah satu ujung resistor 220 Ω .
5. Hubungkan ujung resistor lainnya ke jalur GND pada board. Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar.

2.4 Kode Program

```
const int ledPin = 5;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  Serial.println("Praktikum 1 - Digital Output Dimulai!");
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED Menyala");
  delay(1000);

  digitalWrite(ledPin, LOW);
  Serial.println("LED Mati");
  delay(1000);
}
```

2.5 Cara Kerja Program

Pin GPIO5 dikonfigurasi sebagai OUTPUT pada fungsi setup(). Pada fungsi loop(), pin tersebut diberi nilai HIGH selama 1 detik (LED menyala) kemudian LOW selama 1 detik (LED mati), sehingga LED berkedip secara terus-menerus setiap 1 detik. Status LED juga dicetak ke Serial Monitor sebagai indikator.

2.6 Hasil

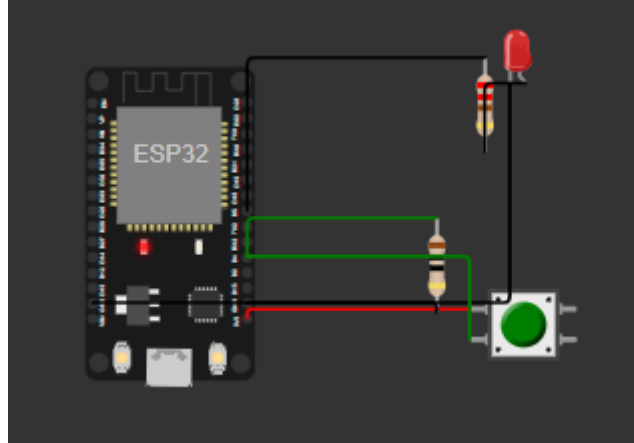
Simulasi berhasil dijalankan menggunakan ekstensi Wokwi Simulator di Visual Studio Code, terintegrasi dengan hasil build PlatformIO (esp32doit-devkit-v1). LED pada simulasi berkedip menyala-mati setiap 1 detik sesuai dengan program, dan pesan “LED Menyala” / “LED Mati” tampil bergantian pada Serial Monitor.

BAB III. PRAKTIKUM 2 — DIGITAL INPUT (MEMBACA PUSH BUTTON)

3.1 Tujuan

Mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol (push button), memanfaatkan rangkaian resistor pull-down eksternal.

3.2 Rangkaian



Gambar 3.1 Rangkaian LED dan push button pada Praktikum 2

3.3 Langkah Kerja

1. Rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah push button pada breadboard, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama push button ke pin 3V3 (3.3V) pada board.
4. Hubungkan kaki kedua push button ke pin D2 (GPIO 4) pada board.
5. Pada kaki kedua yang sama, pasang resistor 10 k Ω , lalu hubungkan ujung lain resistor tersebut ke jalur GND.

3.4 Kode Program

```
#include <Arduino.h>
const int buttonPin = 4;
const int ledPin = 5;
int lastButtonReading = LOW;
int stableButtonState = LOW;
bool ledState = false;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != stableButtonState) {
      stableButtonState = reading;
    }
  }
  if (stableButtonState == HIGH) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

```
    if (stableButtonState == HIGH) {  
        ledState = !ledState;  
        digitalWrite(ledPin, ledState ? HIGH : LOW);  
        Serial.print("LED ");  
        Serial.println(ledState ? "ON" : "OFF");  
    }  
}  
}  
lastButtonReading = reading;  
}
```

3.5 Cara Kerja Program

Pin GPIO4 dikonfigurasi sebagai INPUT dan GPIO5 sebagai OUTPUT. Selama tombol tidak ditekan, resistor pull-down 10 k Ω menarik pin GPIO4 ke kondisi LOW. Saat tombol ditekan, pin GPIO4 terhubung langsung ke 3.3V sehingga bernilai HIGH, yang kemudian membuat program menyalakan LED (`digitalWrite ledPin, HIGH`) dan mencetak pesan “Tombol ditekan! -> LED ON” ke Serial Monitor. Saat tombol dilepas, pin kembali LOW dan LED mati.

3.6 Hasil

Simulasi menunjukkan LED tetap mati ketika push button tidak ditekan (kondisi LOW akibat resistor pull-down). Saat push button ditekan pada simulator, LED menyala dan pesan “Tombol ditekan! -> LED ON” muncul pada Serial Monitor, membuktikan pembacaan input digital berjalan sesuai rangkaian.

BAB IV. LATIHAN

4.1 Latihan 1 — Pemahaman Logika Pin

Soal: Jika rangkaian Praktikum 2 (Active-High / pull-down) dipindahkan ke pin D8 (GPIO 15), lalu tombol ditekan tepat saat board baru diberi daya, apa yang mungkin terjadi?

Jawaban: GPIO15 bukan pin GPIO biasa — pin ini termasuk strapping pin yang digunakan mikrokontroler untuk menentukan mode boot saat baru dinyalakan atau di-reset. Secara default, GPIO15 harus berada pada kondisi LOW tepat saat proses boot supaya board dapat memuat firmware dari flash memory dan menjalankan program seperti biasa. Pada rangkaian pull-down di Praktikum 2, pin dalam keadaan normal (tombol tidak ditekan) memang LOW, sehingga aman. Namun jika tombol ditekan tepat pada saat board baru diberi daya, GPIO15 akan terbaca HIGH pada momen boot tersebut, sehingga bertentangan dengan kondisi yang dibutuhkan bootloader. Akibatnya board berpotensi gagal boot secara normal, masuk ke mode boot yang tidak diharapkan, mengalami crash, atau melakukan reset berulang (boot loop), tergantung seberapa tepat waktu penekanannya bertepatan dengan momen inisialisasi tersebut.

4.2 Latihan 2 — Implementasi Kode (LED Berkedip Asimetris)

Soal: Modifikasi kode Praktikum 1 agar LED menyala cepat 200 ms lalu mati 800 ms.

Jawaban: cukup mengganti dua baris delay() pada fungsi loop():

```
digitalWrite(ledPin, HIGH);  
delay(200);    // sebelumnya delay(1000); -> nyala cepat 200 ms  
  
digitalWrite(ledPin, LOW);  
delay(800);    // sebelumnya delay(1000); -> mati agak lama 800 ms
```

4.3 Latihan 3 — Analisis Sirkuit (LED Terbalik)

Soal: Apa yang terjadi jika kaki LED pada Praktikum 1 dipasang terbalik (Anoda ke GND, Katoda ke D1)? Apakah terjadi error kompilasi? Mengapa LED tidak menyala?

Jawaban: Program tetap dapat dikompilasi tanpa error. Compiler hanya memeriksa kebenaran sintaks dan logika kode (misalnya digitalWrite(ledPin, HIGH)), dan sama sekali tidak mengetahui maupun memeriksa bagaimana komponen fisik dirangkai di breadboard — itu dua hal yang terpisah. LED tidak menyala karena LED adalah komponen dioda yang hanya dapat mengalirkan arus listrik satu arah (forward bias), yaitu dari Anoda menuju Katoda. Ketika Anoda dihubungkan ke GND dan Katoda ke pin D1 (yang bernilai HIGH/3.3V), maka arah tegangan yang diberikan menjadi terbalik (reverse bias) terhadap dioda tersebut. Pada kondisi reverse bias, dioda menghambat aliran arus (hanya melewatkan arus bocor yang sangat kecil), sehingga LED tidak mendapat arus yang cukup untuk menyala.

4.4 Latihan 4 — Pengembangan Algoritma (Logika Terbalik)

Soal: Ubah blok if-else Praktikum 2 sehingga LED menyala saat tombol TIDAK ditekan, dan mati saat tombol DITEKAN.

Jawaban: cukup membalik kondisi pemeriksaan pada blok if, dari == HIGH menjadi == LOW (atau menukar isi blok if dan else):

```
buttonState = digitalRead(buttonPin);  
  
if (buttonState == LOW) {           // sebelumnya: == HIGH  
    digitalWrite(ledPin, HIGH);    // tombol TIDAK ditekan -> LED menyala  
} else {
```



```
digitalWrite(ledPin, LOW);    // tombol DITEKAN -> LED mati  
}
```

BAB V. TUGAS — MODIFIKASI SISTEM SAKELAR TOGGLE (LATCHING)

5.1 Tujuan

Mengimplementasikan logika pelacakan status (state tracking) dan membedakan perilaku tombol momentary dengan latching, sehingga push button berfungsi layaknya sakelar lampu ruangan: sekali ditekan LED menyala dan tetap menyala, ditekan sekali lagi LED mati dan tetap mati.

5.2 Rangkaian

Menggunakan konfigurasi hardware yang persis sama dengan Praktikum 2 (rangkain push button dengan resistor pull-down 10 kΩ pada GPIO4, LED pada GPIO5) — lihat Gambar 3.1.

5.3 Kode Program

```
#include <Arduino.h>
// ===== TUGAS: SAKELAR TOGGLE (LATCHING) =====
// Rangkaian: sama persis dengan Praktikum 2 (push button pull-down 10k)

const int buttonPin = 4; // GPIO4, terhubung ke push button (pull-down)
const int ledPin = 5;    // GPIO5, terhubung ke LED

// --- Variabel GLOBAL untuk melacak status (state tracking) ---
// Variabel ini SENGAJA diletakkan di luar setup()/loop() supaya nilainya
// tidak hilang/reset setiap kali loop() dijalankan ulang (loop() berjalan
// ribuan kali per detik). Dengan begini sistem 'ingat' kondisi terakhir
// LED (ON/OFF) dan kondisi tombol pada iterasi sebelumnya.
bool ledState = false; // status LED saat ini: false = OFF, true = ON
int lastButtonReading = LOW; // pembacaan pin mentah pada iterasi sebelumnya
int stableButtonState = LOW; // status tombol yang sudah dianggap stabil

unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50; // jeda stabilisasi (ms)

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
  digitalWrite(ledPin, LOW); // pastikan LED mati saat sistem baru menyala
}

void loop() {
  int reading = digitalRead(buttonPin);

  // --- Debouncing (non-blocking, pakai millis()) ---
  // Kontak fisik push button bisa 'bergetar' (bouncing) beberapa milidetik
  // saat ditekan/dilepas, sehingga pin sempat terbaca berubah-ubah dengan
  // sangat cepat. Setiap kali pembacaan berubah, timer direset; status baru
  // baru dianggap SAH (stabil) kalau sudah tidak berubah selama debounceDelay.
  if (reading != lastButtonReading) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {

    // --- Edge detection ---
    // Toggle LED hanya boleh terjadi SEKALI per satu kali tekan, yaitu tepat
    // saat status stabil berubah (bukan selama tombol ditahan terus-menerus).
```

```

    if (reading != stableButtonState) {
        stableButtonState = reading;

        if (stableButtonState == HIGH) {
            // --- State tracking: balik status LED (latching) ---
            ledState = !ledState;
            digitalWrite(ledPin, ledState ? HIGH : LOW);

            Serial.print("Tombol ditekan -> LED sekarang: ");
            Serial.println(ledState ? "ON (menyala tetap)" : "OFF (mati tetap)");
        }
    }

    lastButtonReading = reading;
}

```

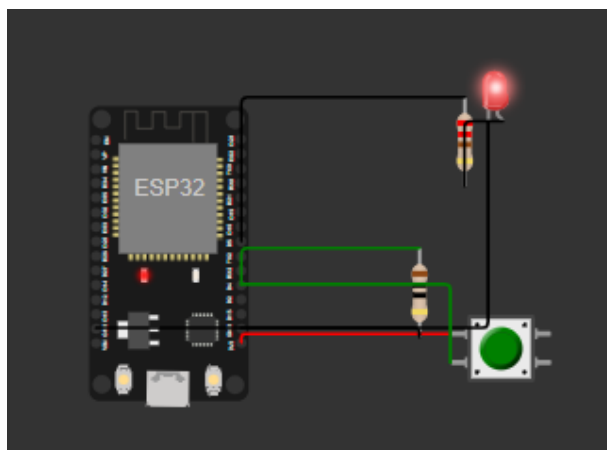
5.4 Penjelasan Logika

Sistem toggle ini berbeda dari Praktikum 2 karena butuh mengingat status LED antar-penekanan, bukan hanya mengikuti kondisi tombol secara langsung. Dua konsep utama yang diterapkan:

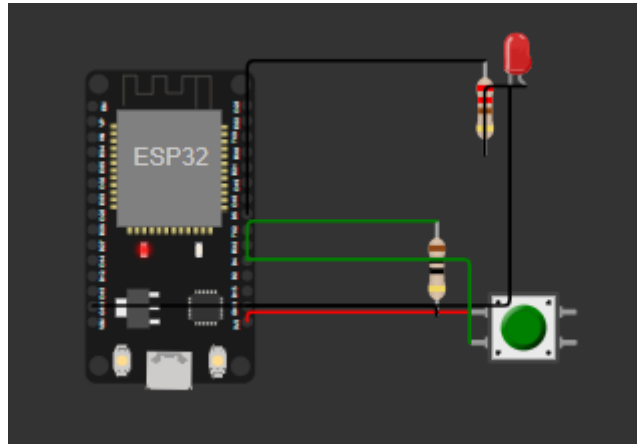
- **State tracking:** variabel `ledState` dan `lastButtonState` dideklarasikan sebagai variabel global (di luar `setup()` dan `loop()`) agar nilainya tetap tersimpan dari satu iterasi `loop()` ke iterasi berikutnya, sehingga program dapat membandingkan kondisi tombol ‘sekarang’ dengan ‘sebelumnya’ dan mengingat status LED terakhir.
- **Edge detection:** perubahan status LED hanya dipicu tepat saat terjadi transisi tombol dari LOW ke HIGH (`reading == HIGH && lastButtonState == LOW`), bukan selama tombol ditahan HIGH terus-menerus. Ini yang membuat efeknya latching (nempel) alih-alih momentary.
- **Debouncing:** `delay(200)` singkat dipasang tepat setelah transisi terdeteksi, untuk melewati periode getaran mekanis (bouncing) kontak tombol sehingga satu kali tekan fisik tidak terbaca sebagai beberapa kali penekanan yang menyebabkan LED flicker atau toggle ganda.

5.5 Hasil Pengujian

Berdasarkan pengujian pada simulator Wokwi (VSCode): saat sistem baru dinyalakan, LED dalam keadaan mati. Tombol ditekan sekali (klik lalu lepas) → LED menyala dan tetap menyala meski tombol sudah dilepas. Tombol ditekan sekali lagi → LED mati dan tetap mati. Tidak ditemukan gejala flickering maupun toggle ganda selama pengujian, dan pesan status “LED ON” / “LED OFF” tercetak konsisten satu kali per penekanan pada Serial Monitor.



Gambar 5.1 Tampilan Pengujian LED Menyala dengan sekali klik



Gambar 5.2 Tampilan Pengujian LED Mati dengan Sekali Klik

BAB VI. KESIMPULAN

- ESP32 dapat mengendalikan komponen output digital seperti LED dengan fungsi `pinMode()` dan `digitalWrite()` melalui pin GPIO yang dikonfigurasi sebagai OUTPUT.
- ESP32 dapat membaca status komponen input digital seperti push button dengan fungsi `digitalRead()` melalui pin GPIO yang dikonfigurasi sebagai INPUT, dengan bantuan resistor pull-down agar kondisi LOW terdefinisi dengan jelas saat tombol tidak ditekan.
- Perilaku tombol momentary (LED aktif selama ditekan) dan latching (LED berpindah status permanen setiap kali ditekan) dibedakan oleh ada tidaknya mekanisme edge detection dan state tracking pada variabel global dalam program.
- Kombinasi PlatformIO dan ekstensi Wokwi Simulator di Visual Studio Code memungkinkan proses pemrograman dan simulasi rangkaian mikrokontroler ESP32 dilakukan dalam satu editor tanpa memerlukan perangkat keras fisik.